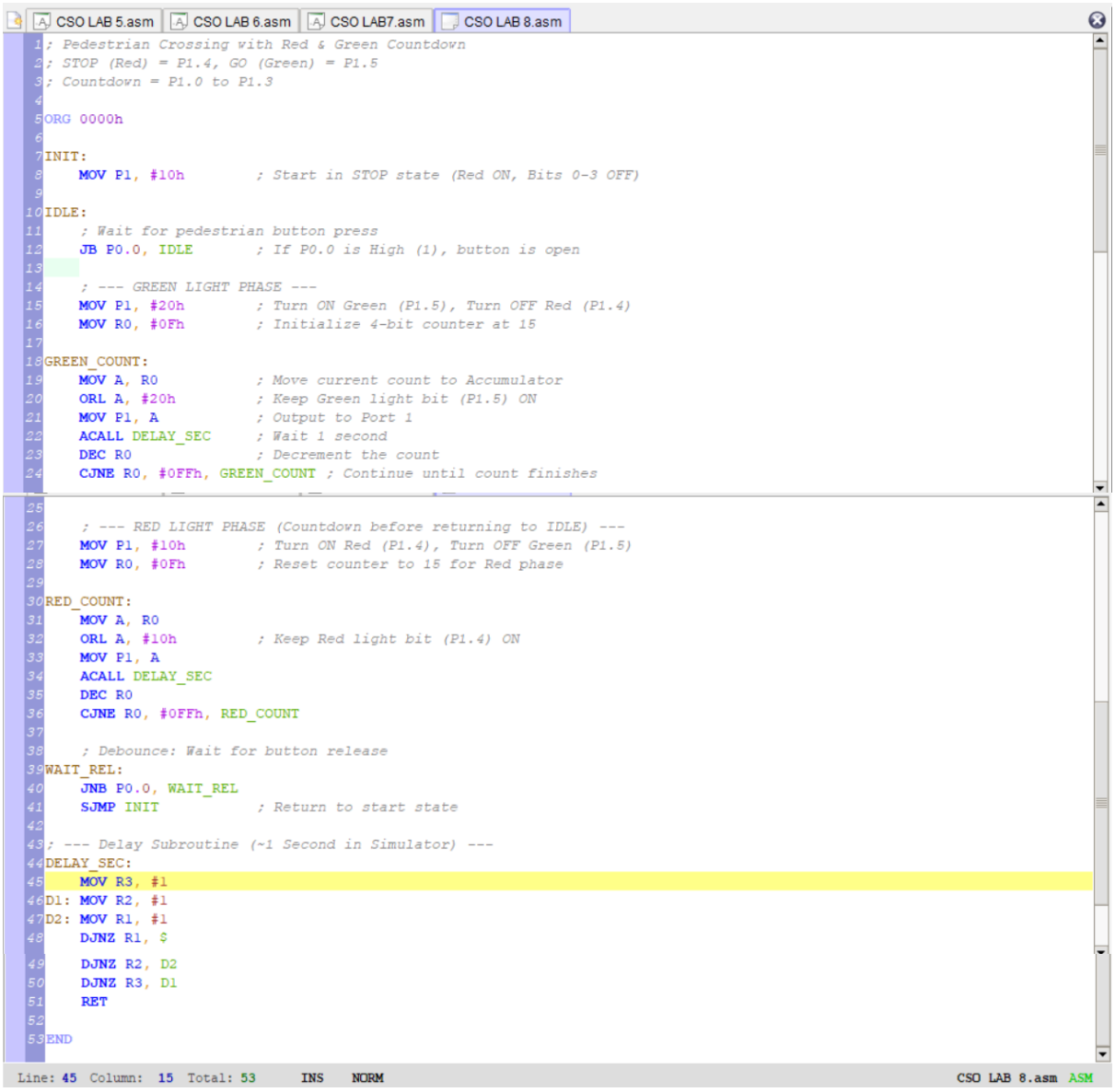
CSO Lab 8 : State Machine Design with 8051

Question: Write a program that replicate a pedestrian crossing light with one button input. The system will be in STOP state until a button is pressed. After a delay, the state will transit to GO state and will stay in this state for a longer delay. Then the system will transit back to STOP state.



Logic Breakdown:

1. Initialization and Idle State

- Startup Configuration: The program begins by forcing the Accumulator and Port 1 to a specific state, typically turning on the Red LED (P1.4) to signify that traffic is stopped.
- The Wait Loop: The system enters an "Idle" state using the JB P0.0, IDLE instruction. Because of the "Open = 1" logic in the 8051, the program traps itself in this loop as long as the button is not pressed.

2. The Green Light (GO) Phase

Once the button is pressed (pulling P0.0 to 0), the program breaks out of the idle loop and enters the Green Phase:

- Signal Switch: It immediately clears the Red light bit and sets the Green light bit (P1.5).
- Loading the Counter: A register (like R0) is loaded with the value 0Fh (decimal 15) to start the 4-bit countdown.
- Simultaneous Output: During the countdown loop, the program uses the ORL (Logical OR) instruction. This is critical because it merges the binary count (bits 0–3) with the Green light signal (bit 5), ensuring the Green light stays lit while the LEDs count down.

3. The Red Light (STOP) Phase

After the Green countdown reaches zero, the system transitions to the Red Phase to warn pedestrians or motorists that the cycle is resetting:

- Signal Switch: The Green light is turned off, and the Red light (P1.4) is turned back on.
- Secondary Countdown: Similar to the Green phase, the counter is reset to 15. The ORL instruction is used again, this time with #10h to keep the Red light active during the 4-bit binary countdown.

4. Timing and Roll-over Logic

- The Delay Subroutine: Because the 8051 operates at high speeds, the program calls a DELAY_SEC subroutine after every count decrement. This uses nested loops to waste clock cycles, creating a visible "1-second" pause between numbers.
- Decrement and Jump (CJNE): The program uses DEC R0 to reduce the count. It checks for completion by comparing the register to FFh. In 8bit logic, after a register reaches 00h, the next decrement causes it to "roll over" to FFh, which signals the program to exit the current countdown loop.

5. Debouncing and Reset

- Button Release: At the end of the Red countdown, the WAIT_REL loop prevents the system from immediately restarting if the user is still holding the button down.
- Return to Start: The SJMP INIT instruction sends the program back to the very beginning to wait for the next pedestrian request.

Demo Video

https://drive.google.com/drive/folders/1rnfQfxfr03Xsfzd2DmIeLebdBVp3SIGy